Spec-Driven, SLO-Aware Agents on a Single GPU

Michael Maloney
Neuralift
University of New Hampshire
mike.maloney@unh.edu

Abstract

When you ship an LLM agent, nobody cares about your benchmark score. What matters is whether the agent returns valid JSON your downstream code can parse, whether its claims are actually grounded in the documents you fed it, and whether it responds before users give up waiting. These are not hypothetical concerns—they are the failure modes that cause 3am pager alerts, erode user trust, and block production deployments after months of development. We built *SpecSLOEval* because existing evaluation tools ignore this reality—they measure accuracy in isolation, detached from the contracts and deadlines that define production success.

SpecSLOEval treats JSON Schemas as enforceable contracts, compiles them to backend-specific decoding constraints, and measures what actually breaks deployments: structural failures, unfaithful claims, flaky outputs across runs, and tail latency that blows past SLO budgets. The uncomfortable finding from our single-GPU experiments: you can achieve 99.7% schema validity and still have 0% Success@SLO under a 1.2-second deadline. Perfect structure, zero deployability. This is why we argue for Success@SLO—quality gates *and* deadline—as the only honest deployment metric.

1 Introduction

Here is the uncomfortable truth about deploying LLM agents: the demo that impressed your stakeholders will fail in production for reasons that never show up in academic benchmarks.

The agent that scored 87% on your evaluation set will occasionally emit malformed JSON that crashes your ETL pipeline at 3am. It will confidently cite numbers that appear nowhere in the documents you retrieved. It will sometimes take 8 seconds to respond when your SLO budget is 1.2 seconds, and that tail latency will propagate through your system until users start complaining. These are not edge cases—they are the default behavior of agents that were never evaluated against the contracts and deadlines that actually define production success.

We focus specifically on single-GPU deployments because this is where most teams actually run agents. Not everyone has a fleet of A100s. Many organizations run open-weight models on an RTX 4090 or M2 Max behind LM Studio or Ollama, partly for cost, partly to keep sensitive data on-premise. These setups face all the same production pressures as cloud deployments—schema contracts, faithfulness requirements, latency SLOs—but with tighter resource envelopes and more visible tail behavior.

The research community has attacked pieces of this problem. Serving systems like vLLM [1] and SGLang [2] optimize throughput and tail latency, but treat the model as a black box emitting unconstrained text. Structured output tools [3, 4] improve syntactic validity, but evaluation stops at "does it parse?" What nobody has built is a framework that measures the full stack: structure *and* accuracy *and* faithfulness *and* stability *and* latency, all evaluated together against explicit contracts and deadlines.

This is why we built SpecSLOEval. The framework treats JSON Schemas as enforceable contracts, compiles them to whatever constraint mechanism your backend supports (OpenAI-style schemas, GBNF grammars, or just validation-and-retry), and evaluates everything through a single `criteria.yaml` file that defines what "good enough" means for your deployment. We designed the metrics to double as reward signals for reinforcement learning [5, 6], because evaluation without a path to improvement is just documentation of failure.

What we contribute. Three things, in order of how much they will change your workflow:

- **A spec compiler that actually works across backends.** Hand us a JSON Schema; we will turn it into whatever your serving stack can enforce, then validate the output anyway because we do not trust the enforcement to be complete. If validation fails, we retry with bounded budget. All of this gets reflected in your latency measurements, because pretending retries are free is how you end up surprised in production.
- **Success@SLO as your deployment metric.** We define this as the fraction of requests that pass your quality gates *and* finish before your deadline. Not accuracy alone. Not latency alone. The joint distribution, because that is what your users experience.
- **Honest measurements from a single GPU.** We ran the full configuration ladder—unconstrained, provider-structured, validation-and-retry, grammar-constrained, self-consistency—on real hardware you can buy for under $4,000. No cluster, no cloud dependency, no hidden inference costs. The results are uncomfortable but reproducible.

Claim A1 (Quality–SLO coupling). Even with strong contract enforcement, tail latency can dominate deployment outcomes. In a 2,000-episode long-context stress test, we observe **99.7%** schema-valid outputs yet **0.0** Success@SLOunder a 1200 ms budget—because the latency distribution shifts beyond the p95envelope. This illustrates the coupling we target: mechanisms that improve structural correctness can still fail service constraints unless evaluated end-to-end.

Paper organization. Section 2 reviews related work in LLM serving, constrained decoding, and agent evaluation. Section 3 formulates contract-first, SLO-aware serving as constrained optimization over configurations. Section 4 details our spec-to-decoder compilation and serving-time hardening (validation-and-retry, budgeted selection). Section 5 presents the evaluation framework and test families, including faithfulness scoring and trajectory checks. Section 6 reports single-GPU experiments and decoding-mode comparisons. Section 7 discusses limitations and future directions.

2 Background and Related Work

Four bodies of work are relevant here, and all of them are incomplete in ways that matter for production deployments.

2.1 Tail Latency and LLM Serving

The SRE community has known for years that users experience the tail, not the mean [7]. If your p99 is 8 seconds, you have a latency problem even if your average is 200ms. LLM serving makes this worse because output lengths vary wildly, batching creates contention, and autoregressive decoding is inherently sequential.

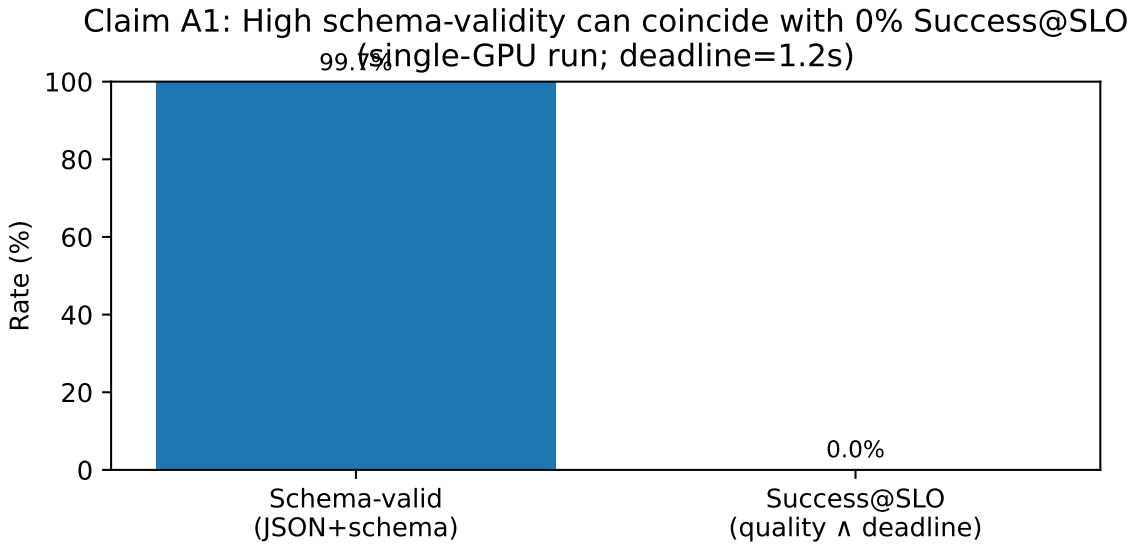


Figure 1: Claim A1 in one picture. In our current single-GPU long-context run, schema-validity is **99.7%** while Success@SLOis **0.0%** under a 1200 ms deadline. Structural correctness alone is therefore an insufficient deployment gate; the correct operational metric is joint: *quality gates* $\wedge$ *deadline*.

The serving systems community has responded with impressive infrastructure. vLLM's PagedAttention and continuous batching [1] are now standard. SGLang [2] optimizes structured programs. FlashAttention [8] cuts memory and compute. Sarathi-Serve [9] tackles scheduling under realistic traffic.

But here is what they all assume: that the model emits unconstrained text and someone else's problem is figuring out whether that text is correct. None of them measure schema validity, faithfulness, or stability. They give you fast inference; they do not tell you if the output is usable.

2.2 Structured Outputs, Grammars, and Contract-First Decoding

If your agent talks to other software, it needs to emit structured outputs. JSON reports. Typed tool arguments. Database records. Free-form text is useless downstream.

The tooling has gotten much better. OpenAI's structured outputs API [4] constrains generation to match a schema. Local servers like LM Studio and Ollama support similar features. Grammar-based decoding can enforce arbitrary formats. JSONSchemaBench [3] provides systematic evaluation, and the results are encouraging: constraints dramatically improve syntactic validity.

But "does it parse?" is a low bar. None of these evaluations ask: how much latency does the constraint add? How often do you have to retry? Does syntactic validity correlate with semantic correctness, or do you just get valid-but-wrong JSON faster? These questions matter for production, and nobody is measuring them systematically.

2.3 Automated Evaluation of Faithfulness and Factuality

Hallucinations are the silent killer of agent deployments. Your model will confidently state that revenue increased 15% when the source document says 12%. It will cite a paragraph that does

not exist. It will invent plausible-sounding statistics. And unless you are checking every output manually, you will not notice until someone makes a bad decision based on it.

RAGAS [10] and similar LLM-as-judge frameworks try to automate this checking. You ask a second model: "Is this claim supported by this context?" Atomic-fact decomposition [11] goes further, breaking outputs into individual claims and scoring each one.

LLM-as-judge methods are powerful but imperfect. Studies document judge bias and sensitivity to prompt framing and verbosity, motivating careful calibration and auditability [12]. For production use, judge outputs must be treated as *signals* that are logged, inspected, and complemented with task-grounded checks.

Gap: faithfulness evaluation is often bolted on after generation and is rarely integrated with structured-output contracts, tool traces, stability, and latency SLOs in a single harness. SpecSLOEval integrates judge-based faithfulness scoring as one metric family in a strict, auditable gating pipeline.

2.4 Agent Evaluation Frameworks in Practice

Production agent stacks increasingly adopt evaluation pipelines: golden sets, rubric scoring, trajectory inspection, and regression tests. Cloud ecosystems (e.g., Google's ADK and Vertex AI evaluation guidance) emphasize both final answer quality and trajectory quality (tool use, error handling) and provide templates for evaluation workflows. These resources are operationally valuable, but they are often platform-tied, assume managed model hosting, and under-specify how to combine schema contracts, tool-call validity, faithfulness, stability, and tail-latency constraints into a single, portable protocol.

Gap: existing production frameworks describe the *idea* of agent testing but do not provide a backend-agnostic, single-GPU-friendly criteria-driven harness that makes contracts and tail SLOs first-class. SpecSLOEval provides this explicit criteria layer (`criteria.yaml`) and a repeatable execution harness.

2.5 Single-GPU Local Serving and Open-Weight Models

Single-node deployments—a workstation GPU (e.g., RTX 4090) or laptop accelerators (Apple M-series)—are common for privacy, cost predictability, and rapid iteration. Local OpenAI-compatible servers (LM Studio, Ollama) and high-throughput runtimes (vLLM) make such deployments practical. What is missing is a standardized way to evaluate these stacks as *services*: not just average latency, not just "does JSON parse," but the joint behavior under contracts, tool usage, faithfulness, stability, and tail SLOs.

Gap: the ecosystem provides runtimes and knobs, but not an end-to-end evaluation protocol that treats structured contracts and tail SLOs as co-equal first-class citizens. This motivates our problem formulation in Section 3.

3 Problem Formulation

Figure 2 sketches the single-node setting we target. A generic commercial agent sits between upstream requesters (users, batch jobs, or other services) and downstream consumers (dashboards, workflows, humans). For each request, the agent receives unstructured and structured context (documents, logs, records, prior turns), may call tools (APIs, databases, services), and is expected to emit *structured outputs* such as JSON reports, action recommendations, or tool-call payloads. These outputs do not only surface in a chat window; they are consumed by campaign engines, scoring pipelines, ETL jobs, monitoring dashboards, and human decision-makers. Small model

errors can therefore propagate and compound: malformed JSON can crash a pipeline; a mis-typed field can poison an aggregate; a subtly incorrect statement about a key metric can trigger the wrong business action. At the same time, users and systems experience the agent through latency: if responses frequently violate p95/p99budgets, trust erodes even when mean latency and accuracy look acceptable.

We want a formulation that is: (i) amenable to systems-level analysis (latency distributions, SLO constraints, single-GPU capacity); (ii) fine-grained enough to capture semantics (structure, task accuracy, faithfulness, tool behavior, stability); and (iii) compatible with both static configuration selection and reinforcement-learning-based improvement. We therefore define: an abstract *agent+contract model* (Section 3.1), a *single-node serving environment* with explicit latency-based SLOs (Section 3.2), and a vector of *evaluation metrics* configured via a single criteria file (Section 3.3). Figure 3 summarizes the metric families and their strict gating order; Figure 4 gives an optimization view that supports both static configuration search (Phase I) and constrained RL (Phase II).

Throughout this section we use the term *configuration* for a concrete choice of model, runtime, decoding settings, and contract. The rest of the paper instantiates this formulation for single-GPU deployments (RTX 4090-class or Apple M-series) and a standard set of evaluation tasks.

3.1 Agent, Contract, and Configuration

Episode model. We formalize an *agent invocation* as a single request–response episode. Let $\mathcal{X}$ denote the space of requests. Each request $x \in \mathcal{X}$ may consist of:

- a user instruction or task description (free-form text or a structured spec),
- a multiset of context items $C = \{c_1, \ldots, c_m\}$ such as retrieved documents, KB entries, prior turns, log excerpts, or database records,
- optional metadata such as user profile, locale, device information, or authorization context.

We deliberately keep $\mathcal{X}$ domain-agnostic: the same formalism covers analytics summarization, support automation, code editing, incident response, and other agent tasks.

Many agents are also *tool-using*. Let $\mathcal{T}$ be a catalog of tools (APIs, DB queries, calculators), each with a name and a JSON argument schema. An episode may include a tool/trajectory trace

$$\tau = \big((t_1, a_1, o_1), (t_2, a_2, o_2), \ldots, (t_n, a_n, o_n)\big),$$

where $t_i \in \mathcal{T}$ is the tool name, a_i are structured arguments (typically JSON), and o_i is the tool response (or an error).

The episode terminates with a final structured output $y \in \mathcal{Y}$ (e.g., a JSON object consumed by downstream systems). We evaluate both the final output y and the trajectory τ.

Contracts. The structure of $\mathcal{Y}$ is governed by a *contract* $\mathcal{C}$. In practice, contracts are expressed as JSON Schemas, Pydantic models, protocol buffers, or grammars. We abstract a contract as a predicate

$$\mathcal{C} : \mathcal{Y} \to \{0, 1\},$$

where $\mathcal{C}(y) = 1$ iff y is syntactically well formed (e.g., valid JSON) and satisfies all structural constraints: field presence, types, enum values, bounds, nesting, and any additional invariants encoded in the contract. By *contract-first* we mean that $\mathcal{C}$ is specified *before* the agent is invoked and that decoding/validation is explicitly designed to respect $\mathcal{C}$ rather than relying on best-effort prompting alone.

Policy and runtime. We assume a parametric policy π_θ, typically an LLM with prompts and optionally fine-tuned weights or adapters. The policy is deployed via a serving runtime R (e.g., LM Studio, Ollama, vLLM, or `llama.cpp`) exposing an OpenAI-compatible API. Given a request x, the agent produces a trajectory τ and a final output y.

Modern runtimes support different mechanisms to enforce or approximate the contract $\mathcal{C}$. We therefore distinguish:

- the *abstract contract* $\mathcal{C}$, and
- a *backend constraint artifact* $g(R, \mathcal{C})$ produced by a spec compiler (e.g., OpenAI-style JSON schema payloads, backend-specific tool specs, or GBNF grammars).

The spec compiler is described in Section 4; here we treat $g(R, \mathcal{C})$ as the runtime-level representation of the contract.

Let δ denote decoding and orchestration settings (temperature, top-p, max tokens, retry budget, self-consistency budget k, batching flags, speculative decoding, etc.). A configuration is

$$\kappa = (R, \pi_\theta, \delta, \mathcal{C}).$$

Given κ, the induced episode distribution can be written as

$$(\tau, y) \sim \pi_\theta(\cdot \mid x,\ g(R, \mathcal{C}),\ \delta,\ R).$$

Changing R, δ, or $\mathcal{C}$ changes the *effective policy* even if θ is fixed, which is why we treat them as first-class optimization knobs.

3.2 Serving Environment and Latency SLOs

We assume the agent stack runs on a *single, finite-capacity node* equipped with a GPU (e.g., RTX 4090-class) or a laptop-class accelerator (e.g., Apple M-series). The runtime R performs prefill and decoding, possibly batching multiple requests, managing KV caches, and using techniques such as PagedAttention as in vLLM [1].

End-to-end latency. For each request x served under configuration κ, define end-to-end latency

$$L_\kappa(x) = t_{\text{finish}}(x) - t_{\text{start}}(x),$$

where $t_{\text{start}}(x)$ is when the request enters the agent orchestrator, and $t_{\text{finish}}(x)$ is when a *validated* final output y (or an explicit failure object) is available to downstream consumers. This definition includes orchestration overhead such as tool calls, validation, bounded retries, and (when enabled) self-consistency.

Requests arrive according to a workload model W (arrival process + request distribution). Given κ and W, the induced latency distribution is

$$Q_\kappa^W = \mathcal{L}\big(L_\kappa(x) \mid x \sim W\big),$$

summarized by quantiles

$$\text{p50}(Q_\kappa^W), \quad \text{p95}(Q_\kappa^W), \quad \text{p99}(Q_\kappa^W),$$

and, when relevant, time-to-first-token (TTFT) and throughput (QPS). Following Dean and Barroso [7], p95and p99are treated as first-class service metrics.

SLOs as constraints. Service-level objectives (SLOs) are expressed as bounds on these quantiles and on resource budgets such as output length:

$$\mathrm{p95}(Q_\kappa^W) \le B, \qquad \mathrm{p99}(Q_\kappa^W) \le B', \qquad \text{and} \qquad \texttt{tokens_out} \le M.$$

Throughout, we report latency in milliseconds unless stated otherwise.

(Some deployments also specify hard timeouts and/or TTFT constraints; our framework supports these as additional gates.)

Lexicographic preference ordering. In many enterprise settings, latency is not the primary objective. A common operational preference is lexicographic:

- **Correctness and structure first:** JSON must parse and satisfy $\mathcal{C}$; task accuracy and faithfulness must exceed thresholds.
- **Stability next:** outputs should be stable across runs and seeds.
- **Latency last:** once correctness/stability are satisfied, meet p95/p99budgets.

We encode this ordering directly in our evaluation criteria (Section 3.3) rather than treating latency as just another weighted term in an objective.

Episode-level "good and on time". To report a concise, business-aligned metric, we define an episode-level indicator that combines quality gates with an explicit per-episode deadline:

$$\text{Success@SLO}_\kappa(x) = \mathbf{1}[\text{ All required quality gates pass } \wedge\ L_\kappa(x) \le B_{\text{episode}}\ ].$$

In our experiments we set $B_{\text{episode}} := B$, i.e., the same budget used for the p95constraint. With this choice, Success@SLOis the empirical CDF evaluated at the p95budget *after* the required quality gates are applied. Aggregating this indicator over an evaluation set yields the Success@SLOrate. Separately, we still report distributional tail metrics (p95/p99); Success@SLOis a *joint* success rate, not a substitute for quantiles.

3.3 Metric Families and Criteria Configuration

To reason about agents in a way that is auditable, tunable, and reusable across backends, we define for each episode a metric vector

$$m_\kappa(x, \tau, y) \in \mathbb{R}^K,$$

whose components are grouped into families (Figure 3): *structure*, *task accuracy*, *faithfulness*, *tools/trajectory*, *stability*, and *latency/SLOs*. Section 5 gives concrete definitions and example tests; here we define the family-level intent.

Structure. Metrics that capture whether y is syntactically and contractually well formed: JSON parse success, schema validity under $\mathcal{C}$, and structured error taxonomies (missing fields, extra fields, type mismatches, constraint violations). These are necessary conditions for safe downstream consumption.

Task accuracy. When ground-truth labels exist, we compute task metrics such as macro/micro-F1, EM, accuracy, and pass@k.

Faithfulness. Faithfulness metrics quantify whether claims in y are supported by context C, typically via atomic-statement decomposition with a judge model. We track both a normalized faithfulness score and contradiction rates.

Tools and trajectory. For tool-using agents we measure tool argument validity, tool-call success, retry counts, redundant calls, and deviations from canonical plans when available.

Stability. Stability metrics quantify variability across runs: disagreement under repeated sampling, and variance of key metrics across seeds.

Latency and SLOs. We record end-to-end latency and (when relevant) TTFT and token counts, and compute p50/p95/p99plus Success@SLOunder configured budgets.

A single policy surface: `criteria.yaml`. All metric definitions, thresholds, and the *strict gating order* are configured via `agent_eval/criteria.yaml`. Formally, let $\mathcal{F}$ be the ordered list of families (e.g.,

$$\mathcal{F} = [\text{structure}, \text{accuracy}, \text{faithfulness}, \text{tools}, \text{stability}, \text{slo}]),$$

and for each family $f \in \mathcal{F}$ let $\phi_f(\kappa)$ denote its aggregate score(s) on an evaluation set (e.g., JSON validity rate, macro-F1, contradiction rate, tool success). The criteria file defines thresholds τ_f and a strict gate predicate

$$\text{Pass}_f(\kappa) = \mathbf{1}[\phi_f(\kappa) \text{ meets thresholds } \tau_f]\,.$$

Under **strict lexicographic gating**, a configuration is accepted only if it passes all gates in order:

$$\text{Pass}(\kappa) = \prod_{f \in \mathcal{F}} \text{Pass}_f(\kappa).$$

Operationally, this matches how teams deploy agents: a faster configuration is rejected if it breaks contracts or increases hallucinations, even if its latency is excellent.

Among passing configurations, we optionally compute a scalar ranking score (used for dashboards and for RL reward shaping in Phase II), e.g.,

$$J(\kappa) = \sum_j w_j\, \overline{m}_j(\kappa) \quad \text{with} \quad w_j \text{ chosen so that structure/accuracy/faithfulness dominate latency.}$$

The gating policy makes the evaluation semantics explicit and auditable: thresholds and ordering are versioned and reviewed like any other production contract.

3.4 Offline vs. Online Evaluation

Although W denotes the production workload conceptually, in practice we evaluate on a held-out approximation $\widehat{W}$ built from: (i) logged production requests (where available), (ii) synthetic and adversarially constructed cases, and (iii) standardized benchmarks. Our harness is agnostic to whether requests come from W or $\widehat{W}$; the same metric definitions and criteria apply. The central practical requirement is that improvements under $\widehat{W}$ track improvements under W closely enough to be operationally meaningful.

3.5 Static Configuration Selection and Policy Improvement

Figure 4 presents the optimization view underlying our framework. A configuration $\kappa = (R, \pi_\theta, \delta, \mathcal{C})$, together with a workload/environment, induces episodes and metric vectors $m_\kappa(x, \tau, y)$. Aggregating metrics yields summaries $\phi_f(\kappa)$, pass/fail gates, and latency distributions.

We consider two complementary uses:

Static configuration selection (Phase I). In static selection, θ is fixed and we tune R, δ, and sometimes $\mathcal{C}$. The design problem is to find configurations that pass strict gates and then optimize within that feasible set, e.g.,

$$\max_{\kappa \in \mathcal{K}} \; J(\kappa) \quad \text{s.t.} \quad \text{Pass}(\kappa) = 1.$$

In practice, $\mathcal{K}$ can be explored via grid search, Bayesian optimization, or bandit-style exploration.

Policy improvement under constraints (Phase II). In policy improvement, θ is trainable and we treat parts of the metric vector as rewards and others as costs, yielding constrained RL formulations. For example, define per-episode reward

$$r_\kappa(x, \tau, y) = \alpha_{\text{task}} m_{\text{task}} + \alpha_{\text{faith}} m_{\text{faith}} + \alpha_{\text{json}} m_{\text{json}} - \beta_{\text{lat}} \max(0, L_\kappa(x) - B_{\text{episode}}),$$

and treat violations (e.g., contradiction flags, tool failures, instability) as costs in a constrained MDP [13, 14]. Using policy gradient methods (e.g., REINFORCE) with LoRA adapters [15, 6], we can update θ while enforcing SLO and reliability constraints.

In both regimes, the objects defined above—contracts, compiled constraints, metric families, strict gating, and latency distributions—form the contract-first, SLO-aware backbone for the rest of the paper.

4 Method: Spec Compilation and Serving-Time Hardening

Here is the engineering that makes contract-first serving actually work. The core insight is simple: your JSON Schema is the source of truth, but different backends need different constraint formats. So we compile the schema once and emit whatever artifact your runtime can actually enforce—OpenAI-style `response_format`, tool specs, or GBNF grammars. Then we validate anyway, because we do not trust the enforcement to be complete.

If validation fails, we retry with a bounded budget. If the budget runs out, we return a typed error object that downstream systems can handle. The key principle: *fail closed*. Every request either produces a validated output or a structured failure. No half-parsed JSON, no unconstrained text, no "best effort."

4.1 Spec Compiler

The compiler takes your JSON Schema and produces whatever your backend can actually use. We target three surfaces that most OpenAI-compatible servers expose:

1. **Provider-native JSON schema decoding.** Backends that implement OpenAI-style structured outputs accept a JSON Schema via `response_format` (or an equivalent parameter) and constrain decoding to schema-valid JSON.

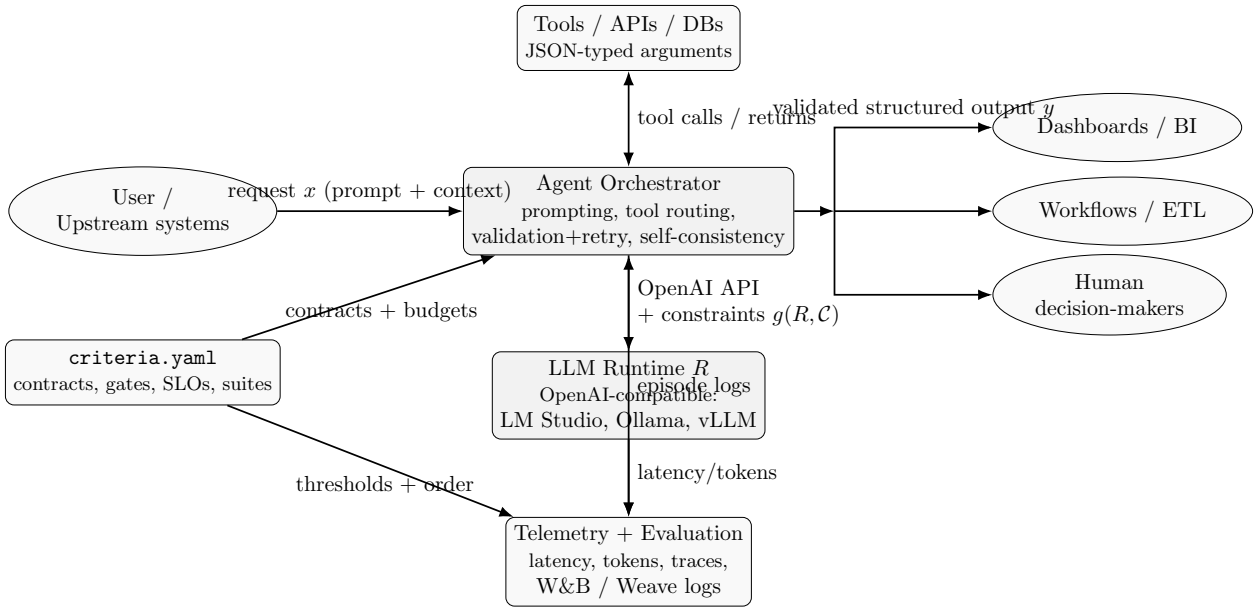


Figure 2: System context for a contract-first, single-node agent. An orchestrator mediates between upstream requests and downstream consumers, calling an OpenAI-compatible runtime and external tools, while enforcing contracts via backend constraints and validation. Telemetry and evaluation log structure/accuracy/faithfulness/tool/stability/SLO metrics under a single versioned policy surface (`criteria.yaml`).

2. **Tool schemas for function calling.** For tool-using agents, each tool $t \in \mathcal{T}$ declares a JSON Schema over arguments. The compiler emits backend tool specifications so the runtime can constrain tool-call argument generation.

3. **Grammar-based constrained decoding.** When schema-native decoding is unavailable or incomplete, the compiler converts $\mathcal{C}$ into a grammar (e.g., GBNF) that can be consumed by grammar-capable backends or wrappers.

The contract $\mathcal{C}$ is held constant across backends; only $g(R, \mathcal{C})$ changes. This is the key to portable evaluation: the same test and validation code applies regardless of whether the backend enforces constraints via schemas, tool specs, or grammars.

Dealing with backend inconsistencies. No two backends support exactly the same JSON Schema features. Some choke on `oneOf`, others ignore `additionalProperties`, and numeric bounds are hit or miss. So the compiler does two things: first, it probes the backend with a small test schema to see what actually works (not what the docs claim). Second, if a feature is unsupported, it either rewrites the schema into something equivalent or marks that feature for post-hoc validation. You get the strongest enforcement your backend can provide, with the validator catching whatever slips through.

4.2 The Serving Wrapper: Trust but Verify

Constrained decoding helps, but it is not enough. Backends are inconsistent. Schemas change. And rare failures still happen. If you only rely on the backend's constraints without explicit validation, you will eventually ship broken JSON to downstream systems and find out at 3am.

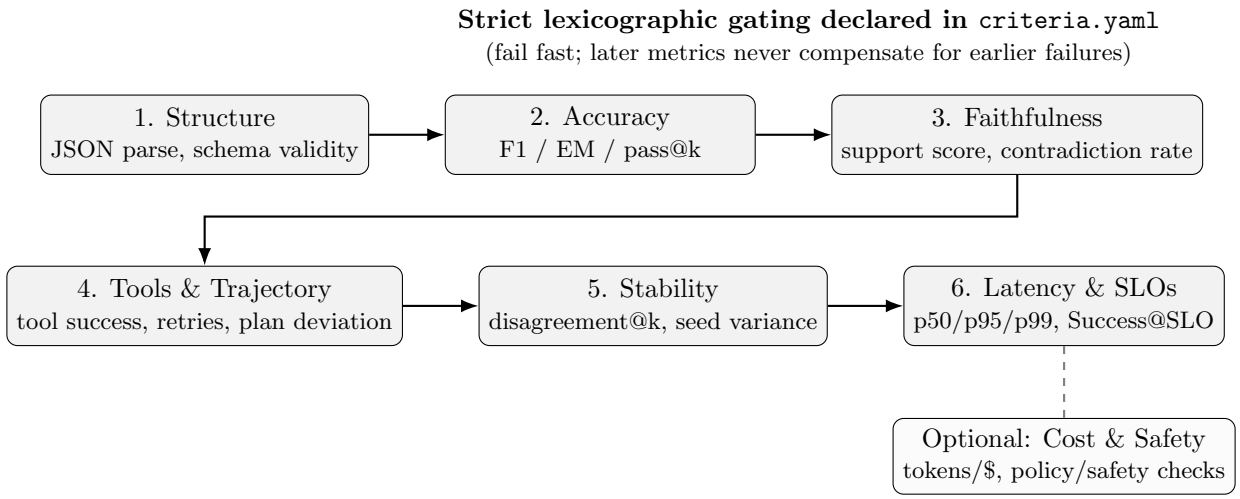


Figure 3: Metric families and strict gating order. We evaluate structure, accuracy, and faithfulness before considering tools/trajectory, stability, and finally SLO adherence. This ordering reflects production preference: a faster agent does not compensate for broken contracts or unfaithful outputs.

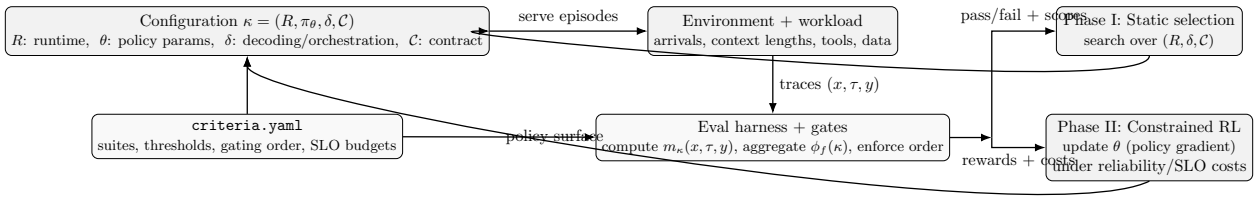


Figure 4: Optimization view. A configuration induces episodes under a workload/environment, which are scored by an eval harness governed by `criteria.yaml`. Phase I selects configurations by searching over runtimes and decoding/orchestration settings; Phase II improves the policy parameters θ under explicit reliability and SLO constraints.

For each model response, the wrapper executes:

1. **Parse:** attempt JSON parsing (or tool-call parsing) to obtain a structured object.
2. **Validate:** validate the parsed object against the declared JSON Schema $\mathcal{C}$ using a reference validator.
3. **Canonicalize:** normalize JSON (stable key order, number/string normalization where allowed, whitespace stripping) and re-validate the canonical form.
4. **Record a taxonomy:** on failure, emit a compact error code and structured details (missing keys, extra fields, type mismatch, enum violation, bound violation).

If the output cannot be validated within budget (Section 4.3), the wrapper returns a *structured failure object* (also governed by a schema) rather than unconstrained text. This makes downstream behavior predictable: failures are typed and machine-actionable.

4.3 Bounded Repair: Fixing Broken Outputs

Most validation failures are fixable. The model emitted JSON with one wrong field type, or it forgot a required property. Instead of failing immediately, you can try again with a targeted repair prompt.

1. Generate under constraints $g(R, \mathcal{C})$.
2. If validation fails, construct a repair prompt containing: (i) the original schema identifier, (ii) a compact error taxonomy, and (iii) an instruction to output *only* a corrected JSON object.
3. Re-run generation with the same contract and a strict retry limit $r_{\max}$.

The critical thing: retries are *budgeted*. They consume your token and wall-clock allowance, and every retry second shows up in your latency measurements. This prevents the infinite-repair failure mode where you keep trying forever and pretend it is free. It is not free. Your users are waiting.

4.4 Self-Consistency: Sample, Score, Select

Sometimes you get better outputs by generating a few candidates and picking the best one. This is especially useful for grounded tasks where faithfulness is brittle—the model might hallucinate on one run but not another. For configurations that enable self-consistency, the wrapper generates up to $k_{\max}$ candidates under a fixed budget: a cap on candidates, a cap on total tokens, and optionally a wall-clock limit.

Each candidate is scored by a lexicographic selector that prefers, in order: (i) schema-valid outputs; (ii) higher task-score proxies when a reference is available; (iii) higher faithfulness/support scores for grounded tasks (using the Phase I judge pipeline); and (iv) fewer tool failures and retries for tool episodes. The selected output is then logged along with all candidates, enabling audit of "why this candidate won".

Why selection stays separate from evaluation. Candidate selection may use heuristic proxies and/or a judge model. To reduce evaluator overfitting, SpecSLOEval can evaluate the final system with a different judge prompt, a different judge model, or additional deterministic checks (Section 7.3). This separation is essential once policy optimization is introduced (Phase II).

4.5 Latency Accounting

The framework measures *downstream-usable* latency: timing starts when the request enters the orchestrator and stops only when a validated output (or typed failure object) is produced. Therefore, $L_\kappa(x)$ includes backend inference time, tool time, validation time, and any bounded repair or candidate selection overhead. This definition aligns latency measurement with operational reality: downstream systems cannot consume partial or invalid outputs.

5 Evaluation Framework and Test Families

Figure 3 summarized the metric families we track. This section describes how SpecSLOEval turns those abstract families into concrete, repeatable tests that can be executed unchanged across single-GPU, OpenAI-compatible backends (LM Studio, Ollama, vLLM, `llama.cpp` wrappers). Our goal is operational: produce evaluation artifacts that are (i) contract-first, (ii) SLO-aware (tail latency, not just averages), and (iii) auditable end-to-end so every aggregate number can be traced back to per-episode inputs/outputs, schema validations, tool transcripts, and (when used) judge-model decisions.

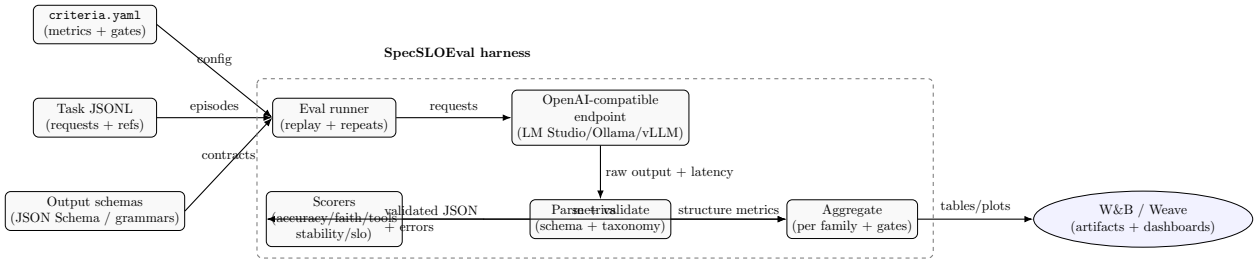


Figure 5: SpecSLOEval evaluation architecture. A single `criteria.yaml` and versioned task/schema artifacts drive an evaluation runner that replays episodes against an OpenAI-compatible backend. The harness performs parsing/validation, computes per-family metrics (including stability repeats and SLO metrics), applies lexicographic gates, and logs all per-episode artifacts and aggregates to W&B/Weave for auditability and reproducibility.

Design principles. SpecSLOEval is built around four principles that matter in production settings:

- **Contract-first by construction.** Every task declares an explicit output schema (JSON Schema or a backend-compatible equivalent). Structural validity is a first-order metric, not a best-effort prompt instruction.
- **Lexicographic gating.** We treat correctness/faithfulness as *gates* and latency primarily as a *constraint*. A configuration that is fast but structurally invalid or unfaithful is rejected.
- **Backend-agnostic measurement.** The harness assumes only an OpenAI-compatible API plus observability hooks (token counts and latency). Everything else is done by the harness: schema validation, error taxonomies, tool mocks, stability repeats, and aggregation.
- **Auditability and reproducibility.** We log per-episode artifacts (inputs, outputs, parsed JSON, validation errors, tool traces, judge prompts/outputs) and experiment metadata (model, runtime, decoding params, schema version, git commit, seeds) to W&B and optionally Weave (`https://docs.wandb.ai/models/quickstart`, `https://docs.wandb.ai/models/evaluate-models`).

5.1 Evaluation Configuration via `criteria.yaml`

The entire framework is driven by a single configuration file, `agent_eval/criteria.yaml`. This file is the *policy* of evaluation: it declares what to measure, what thresholds must hold, and how to aggregate across tasks and metric families. Versioning this file alongside code is non-negotiable for trustworthy comparisons: when thresholds or gates change, the change is explicit and reviewable.

Schema surface. In our baseline design, `criteria.yaml` provides:

1. **Suites and task bindings:** a suite is a named grouping of tasks, where each task references a JSONL file and an output schema.
2. **Gating policy:** an explicit order over metric families (e.g., structure → accuracy → faithfulness → tools → stability → SLO) and a policy (`strict` means earlier failures block later credit).

3. **Thresholds:** per-metric pass thresholds or max violation rates (e.g., schema-validity ≥ 0.99, contradiction rate ≤ 0.05).

4. **SLO budgets:** p95/p99 targets, hard timeouts, and max tokens-out.

Concrete example (aligned with our Phase I suite). For clarity we show a simplified excerpt that matches the structure of the Phase I criteria used in our runs (full file is versioned with the harness):

```
schema_version: "1.0"
criteria_id: "p1_core_v1"

suites:
  p1_core:
    tasks:
      - id: "t1_clinc"
        family: "accuracy"
        task_file: "tasks/clinc_en.jsonl"
        output_schema: "tasks/schemas/clinc_nlu_schema.json"
      - id: "t2_hotpot"
        family: "faithfulness"
        task_file: "tasks/hotpot_dev.jsonl"
        output_schema: "tasks/schemas/hotpot_explainer_schema.json"
      - id: "t3_tools"
        family: "tools"
        task_file: "tasks/t3_tools.jsonl"
        output_schema: "tasks/schemas/t3_tool_call_schema.json"

gating:
  order:
  - structure
  - accuracy
  - faithfulness
  - tools
  - stability
  - slo
  policy: "strict"

structure:
  json_valid: 0.99

accuracy:
  clinc_intent_f1: 0.85
  hotpot_answer_em: 0.65

faithfulness:
  hotpot_faithfulness: 0.80
  hotpot_contradiction_rate_max: 0.05

tools:
  tool_success_rate: 0.90

stability:
  disagreement_max: 0.10
```

```
slo:
  success_at_slo: 0.75
  p95_ms: 1200
  p99_ms: 2000
  max_tokens_out: 512
```

Lexicographic gating (why it matters). Most teams implicitly behave lexicographically ("don't ship broken JSON" → "don't ship hallucinations" → "make it fast enough"), but many evaluation setups do not encode it. We encode it explicitly so that automated comparisons reflect the real risk surface.

5.2 Per-Episode Record and Audit Trail

SpecSLOEval evaluates *episodes*. Each episode produces a structured record (stored as JSON and logged as an artifact) containing:

- **Inputs:** request text, retrieved/context bundle, tool catalog snapshot, schema identifiers.
- **Outputs:** raw model output, parsed JSON (if parse succeeds), tool call transcript.
- **Observability:** latency (end-to-end and TTFT when available), token counts, retries, backend identifiers, decoding params, random seeds.
- **Evaluations:** structure validity flags and error taxonomy; task accuracy components; faithfulness judge artifacts (prompts, extracted claims, support scores, contradiction flags); stability repeats linkage; SLO success flags.

This record is what makes the framework *publishable*: tables and plots can be reproduced by re-aggregating immutable episode artifacts under a fixed criteria version.

5.3 Test Families

We group tests into six families: *Structure*, *Task Accuracy*, *Faithfulness*, *Tools/Trajectory*, *Stability*, and *Latency/SLO*. Each family maps cleanly to real operational risks, which is why we treat them as first-class objects in `criteria.yaml` rather than ad-hoc metrics.

5.3.1 Structure: JSON and Schema Correctness

The structure family measures whether outputs can be consumed by downstream systems without bespoke cleanup. We compute two primary indicators per episode: (i) JSON parse success; and (ii) schema validity under the declared contract $\mathcal{C}$. We additionally record a fine-grained error taxonomy (e.g., missing required fields, additional properties, type mismatches, constraint violations), because aggregate validity alone is insufficient for debugging.

Parameters. Each structural test is defined by:

- a schema/contract $\mathcal{C}$ (name + version),
- prompts/tasks that exercise the schema (including adversarial formatting cases),
- pass thresholds such as `json_valid` ≥ 0.99.

Family	What it measures	Typical failures caught
Structure	JSON parse + schema validity + error taxonomy	malformed JSON, missing/extra fields, type mismatches, invalid enums
Accuracy	label correctness (F1/EM/acc/pass@k)	wrong intent/answer, inconsistent field extraction
Faithfulness	support of claims against context (judge + atomic facts)	hallucinated statements, numeric sign errors, contradictions
Tools/ Trajectory	tool-call validity, success, plan quality, redundancy	invalid tool args, tool thrashing, redundant calls, wrong sequencing
Stability	disagreement across repeats/seeds under fixed config	nondeterministic outputs, brittle prompts/decoding, unstable scores
Latency/SLO	p50/p95/p99 latency, Success@SLO, violation rates	tail-latency collapse, timeouts, throughput collapse under load

Table 1: Metric families and the operational risks they are designed to detect. SpecSLOEval evaluates agents as services: structure and faithfulness failures are treated as deployment blockers; latency is treated primarily as a constraint.

Example S1: Basic JSON/Schema Validity. Fix a moderate schema $\mathcal{C}_{\text{basic}}$ (nested objects, bounded arrays). Run on N prompts and compute

$$m_{\text{json}} = \frac{\#\text{episodes with parse \& schema valid}}{N}.$$

We gate with $m_{\text{json}} \geq 0.99$ for the Phase I suite because rare structural failures are operationally catastrophic.

Example S2: Schema Evolution Robustness. Given $(\mathcal{C}_1, \mathcal{C}_2)$ where $\mathcal{C}_2$ adds optional fields or relaxes bounds, we test: (i) regression in validity when switching schemas; and (ii) whether the agent can emit outputs valid under both (when that is a requirement for rolling upgrades).

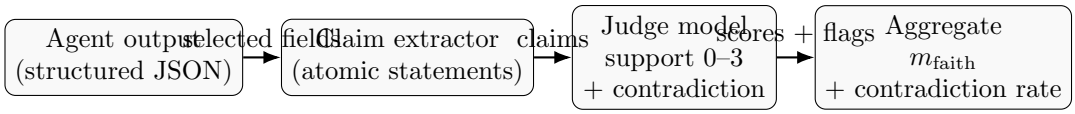
5.3.2 Task Accuracy: F1, EM, and pass@k

The accuracy family measures task correctness against references. We intentionally keep accuracy metrics *separate* from structure metrics: a run can be structurally perfect and still wrong.

Parameters. Accuracy tests specify:

- a dataset $\mathcal{D}_{\text{acc}} = \{(x_i, y_i^{\text{ref}})\}$,
- a schema $\mathcal{C}$ for predictions,
- a scoring map from predicted JSON fields to labels/answers,
- pass thresholds (e.g., intent F1 ≥ 0.85, QA EM ≥ 0.65).

Example A1: Field-Level Extraction F1. For extraction/classification tasks we compute macro-F1 across the evaluated JSON fields. This directly measures whether the agent can reliably populate structured fields, not merely produce valid JSON.



We score only claims that are intended to be factual/grounded (e.g., explanations, key points). The judge receives the original context C and must cite support implicitly via the 0–3 scale; contradictions are tracked as a separate hard signal.

Figure 6: Faithfulness evaluation pipeline in SpecSLOEval. We decompose outputs into atomic statements and score each against context using a judge model. The aggregate faithfulness score and contradiction rate are used for gating and are logged with full judge prompts/outputs for auditing.

Example A2: Programmatic QA Exact Match and pass@ k. For QA-style tasks we embed the answer in a simple schema (e.g., `final_answer`) and compute EM/F1. When sampling is enabled, pass@ k measures whether any of k samples matches the reference.

5.3.3 Faithfulness: LLM-as-Judge with Atomic Statements

Faithfulness measures whether the agent's statements are supported by the provided context C. We operationalize this using an LLM-as-judge pipeline inspired by RAGAS [10] and atomic-fact approaches [11], but with explicit structure:

1. Extract *atomic statements* from the agent output (restricted to designated fields, e.g., a narrative explanation field).
2. For each statement, ask a judge to score support on a 0–3 scale and flag contradictions.
3. Aggregate into a normalized faithfulness score and a contradiction rate, both used for gating.

Scoring. Let S be the set of extracted statements and $s_j \in \{0, 1, 2, 3\}$ the judge support score for statement j with contradiction flag $c_j \in \{0, 1\}$. We define an aggregate (one reasonable instantiation) as:

$$m_{\text{faith}}(x, y) = \max\left(0,\ \frac{1}{|S|}\sum_{j \in S}\frac{s_j}{3}\ -\ \lambda_{\text{contr}} \cdot \frac{1}{|S|}\sum_{j \in S} c_j\right),$$

with λ_{contr} set so that even a small contradiction rate meaningfully depresses the score. We also gate on a maximum contradiction rate (e.g., ≤ 0.05) because contradictions are qualitatively worse than omissions.

Example F1: Cited Analytics Summaries. The agent writes a short summary and key points from retrieved documents. The judge decomposes these into claims and scores them. We require $m_{\text{faith}} \geq 0.75$ and contradiction rate $\leq 5\%$ on the Phase I cited-summary suite (T1–T2).

Example F2: Numeric and Trend Consistency. When contexts include metrics/time-series, we explicitly prompt the judge to focus on directionality and magnitude. This catches the most damaging real-world hallucinations in analytics/monitoring agents: sign inversions, fabricated deltas, and exaggerated trends.

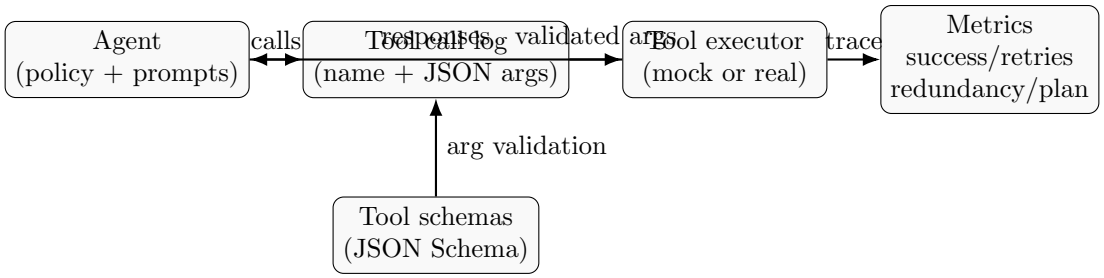


Figure 7: Tool and trajectory evaluation. We validate tool arguments against declared schemas, execute tools (mocked deterministically for reproducibility or real services when available), and compute tool success and trajectory efficiency metrics from the full trace.

5.3.4 Tools and Trajectories

For tool-using agents, final output quality is insufficient: operational reliability depends on whether tool calls are valid, succeed, and are not wasteful. We evaluate both *tool-call validity* (arguments satisfy the tool schema) and *trajectory quality* (no thrashing, minimal redundancy, appropriate sequencing).

Parameters. Tool tests specify:

- a tool catalog (tool names, descriptions, JSON argument schemas),
- an executor (deterministic mock for CI; optionally real services in staging),
- optional canonical plans (minimal tool sequences),
- thresholds (e.g., tool success ≥ 0.90; redundant calls bounded).

Example T1: Tool-Argument Validity and Success Rate. We validate each tool call against its argument schema; the executor returns success/failure deterministically. We compute

$$m_{\text{tool}} = \frac{\#\text{successful tool calls}}{\#\text{tool calls}},$$

and gate on $m_{\text{tool}} \geq 0.90$.

Example T2: Trajectory Cost, Redundancy, and Plan Deviation. When a canonical plan exists, we compute edit distance between observed and expected tool sequences, plus counts of extra calls and invalid retries. These metrics are operationally important because tool thrashing inflates tail latency and can trip downstream rate limits.

5.3.5 Stability Across Runs

Stability is the most under-measured production risk: unstable structured outputs break monitoring, A/B comparisons, and trust. We define stability tests by repeating identical episodes under fixed configuration and measuring output disagreement.

Parameters. Stability tests specify:

- a prompt set $\mathcal{D}_{\text{stab}}$ representative of the workload,
- a number of repeats k per input (e.g., $k = 5$ or $k = 20$),
- an equivalence rule: exact JSON equality, canonical JSON equality, or claim-overlap equivalence.

Example ST1: Disagreement@ k for Structured Outputs. Run k times per input. Let $\text{Eq}(y^{(a)}, y^{(b)})$ be equivalence under canonicalization. Define Disagreement@ k as

$$\text{Disagree@}k = 1 - \frac{\#\text{runs matching the modal output}}{k},$$

and gate on Disagree@$k \leq 0.10$ (as in our Phase I criteria) for stability-sensitive deployments.

Example ST2: Seed Variance on Accuracy and Faithfulness. We evaluate across multiple seeds and compute variance in task score and faithfulness. High variance indicates brittle behavior even if the mean score is acceptable.

5.3.6 Latency and SLO Tests

Latency/SLO tests measure whether configurations satisfy tail-latency commitments under realistic request mixes and (optionally) load. We focus on p95 and p99 because these are the operational pain points in multi-step agent flows, consistent with "The Tail at Scale" [7] and modern LLM serving work [1, 9].

Workloads. We parameterize workloads by:

- context length distribution (short vs long contexts),
- output token budgets,
- arrival process (steady Poisson; bursty sequences),
- tool-use frequency (for tool-heavy agent mixes).

Example L1: Success@SLOunder Mixed Load. For each episode we compute quality metrics (structure/accuracy/faithfulness/tools/stability where applicable) and latency $L_\kappa(x)$. We then define Success@SLOas:

$$\text{Success@SLO}_\kappa(x) = \mathbf{1}\{\text{quality gates pass}\} \cdot \mathbf{1}\{L_\kappa(x) \leq B\},$$

and aggregate to get a business-aligned "good and on-time" rate.

Example L2: QPS vs p95 Curves and Configuration Frontier. We sweep QPS levels and plot QPS vs p95/p99 and QPS vs Success@SLO, producing an operating frontier for each configuration. This makes tail collapse visible and supports principled choices of token budgets, batching, and self-consistency depth.

5.4 Extensibility, Logging, and Use in RL

Extending the metric space. SpecSLOEval is intentionally modular. Adding a metric means: (i) adding a key and threshold in `criteria.yaml`; and (ii) implementing a scorer function that consumes the per-episode record. This supports domain metrics (e.g., "invoice total reconciles," "campaign budget constraint satisfied") without rewriting the harness.

Logging and auditability. All runs are logged to W&B, including:

- per-episode artifacts (inputs, outputs, parsed JSON, error taxonomy, tool traces, judge artifacts),
- per-run configuration (backend, model id, decoding params, schema versions, criteria version),
- aggregated tables/plots (per-family metrics, gating pass/fail, latency quantiles, Success@SLO).

We optionally instrument judge pipelines and multi-stage scoring in Weave so that intermediate computations can be inspected and replayed.

Bridging evaluation and optimization (Phase II). The same metric vector that gates deployments also defines reward/cost surfaces for SLO-aware RL. Concretely, we can define a scalar reward for an episode by combining metric components:

$$r(x, y) = \sum_k w_k \, m_k(x, y) \; - \; \beta \, \max(0, L(x) - B),$$

while treating contradiction rate, repeated schema failures, or SLO violations as explicit costs in constrained RL [13, 14]. This design minimizes the common failure mode where "training metrics" and "deployment metrics" diverge.

In Section 6 we instantiate these test families on single-GPU backends and demonstrate how spec-driven decoding and bounded retry/self-consistency shift the trade-off frontier between structural correctness, faithfulness, stability, and tail latency.

6 Experiments

We ran the full framework on hardware you can actually buy: an RTX 4090 desktop and a MacBook Pro. The question we wanted to answer: does constraining decoding (schemas, grammars, validation-and-retry) actually improve outcomes, or does it just add latency?

The short answer: constraints work, but the details matter. Schema-driven decoding eliminates most structural failures. Validation-and-retry catches the rest. But tail latency increases, and some combinations of constraints actually hurt task accuracy. The only way to know what works for your setup is to measure everything together.

This section walks through:

- Hardware, backends, and models (Section 6.1)
- The task suite: intent classification, grounded QA, and tool calls (Section 6.2)
- The configurations we compare: unconstrained, schema-driven, grammar-constrained, with and without validation-and-retry (Section 6.3)
- What we measured and how we report it (Section 6.4)
- The results (Sections 6.5–6.7)

Component	RTX 4090 desktop	M2 Max laptop
Accelerator	RTX 4090 (24GB)	M2 Max (unified memory)
OS	Ubuntu 22.04	macOS 15+
Primary runtime	LM Studio server	LM Studio server
Alt. runtimes	Ollama, vLLM	Ollama (optional)
Primary usage	throughput / tail latency	portability / local dev

Table 2: Hardware/runtime matrix for the single-node regime targeted in this paper.

6.1 Experimental Setup

Hardware. We ran on two machines that represent what most teams actually use:

- **Desktop:** Alienware R15 with RTX 4090 (24 GB VRAM), 64 GB RAM, Ubuntu 22.04, CUDA 12.x.
- **Laptop:** MacBook Pro M2 Max (64 GB unified), macOS 15.

Neither is exotic. You can buy the desktop for about $4,000. The laptop is standard-issue in many engineering organizations.

Serving backends. All backends are exercised through an OpenAI-compatible HTTP interface. Our primary runtime is LM Studio's OpenAI-compatible server (`https://lmstudio.ai/docs/developer/openai-compat/tools`). We additionally evaluate Ollama (`https://docs.ollama.com/capabilities/structured-outputs`) and a vLLM-backed server (`https://arxiv.org/abs/2309.06180`) to expose differences in batching, KV-cache behavior, and tail latency under load. The spec compiler (Section 4.1) emits backend-specific constraint artifacts (JSON Schema, tool specs, or grammars) while keeping the *contract* constant.

Models. We focus on locally served, OpenAI-compatible models that are practical in the single-GPU regime. Our model selection prioritizes four criteria:

- **Geographic diversity:** China (Alibaba), France (Mistral AI), USA (Meta, Google)—demonstrating global applicability.
- **Company diversity:** All major open-source players represented.
- **Size ladder:** 3B → 4B → 8B → 12B—showing scaling behavior within single-GPU constraints.
- **Hardware accessibility:** All trainable on a single RTX 4090 with QLoRA—reproducible on commodity hardware.

The four models used throughout this paper:

1. `Qwen/Qwen3-4B-Instruct` (Alibaba, July 2025)—4B parameters
2. `mistralai/Ministral-3-8B-Instruct` (Mistral AI, Dec 2025)—8B parameters
3. `meta-llama/Llama-3.2-3B-Instruct` (Meta, Sept 2024)—3B parameters
4. `google/gemma-3-12b-it` (Google, March 2025)—12B parameters

(We treat these as interchangeable OpenAI-compatible endpoints; the framework is agnostic to the model family.)

For every run we log: (i) model identifier and revision; (ii) quantization and precision mode (e.g., BF16 vs quantized); (iii) maximum context length and maximum output tokens; (iv) decoding parameters and constraint mode.

Reproducibility. We fix random seeds, log everything to W&B, and version datasets and schemas as artifacts. Before each run we do 20 warmup requests to stabilize KV-cache allocation. Latency is measured client-side, end-to-end, including retries and validation. We report full distributions (p50/p95/p99), not just averages, because averages hide tail behavior.

6.2 The Task Suite

We evaluate on three tasks that represent common production patterns: classification with a large label space, evidence-grounded QA that requires faithfulness, and tool calling with typed argument schemas. All tasks emit structured JSON. All JSON is validated against schemas.

T1: Intent Classification (CLINC150). The classic "what did the user want?" problem. 150 intent classes plus out-of-scope detection, 500 examples. Output schema requires an `intent` field (must match the vocabulary) and optional `confidence`. We measure schema validity, accuracy, and macro-F1. The stress test here is the large label space: 150 valid values, and the model has to pick exactly one.

T2: Evidence-Grounded QA (HotpotQA). Multi-hop question answering where the model must reason across multiple paragraphs, 1,000 examples. Output schema requires an `answer`, a `reasoning` field, and `supporting_facts` (which paragraphs the answer is based on). We measure exact match, F1, schema validity, and—critically—*faithfulness*. Faithfulness is scored by an LLM-as-judge that decomposes outputs into atomic claims and checks each one against the source context [10, 16]. The stress test: can the model avoid making stuff up?

T3: Tool Calling. Agents that do things need to call tools with correctly typed arguments. 500 synthetic examples covering database queries, calculations, and API calls. Output schema enforces tool name validation and argument typing: integer ranges, enums, nested objects. We measure tool-call success (right tool + valid arguments), schema compliance, and trajectory stability. The stress test: can the model follow a function signature?

6.3 Baselines and Configurations

We compare six decoding configurations that progressively add contract enforcement and reliability mechanisms. All configurations share the same underlying policy model unless explicitly noted.

Shared decoding knobs. To ensure comparability, we fix the following unless explicitly swept: temperature, top-p, max output tokens, and request formatting (system/prompt template). When a backend supports `response_format` or tool schemas, we pass the *same* JSON Schema used by the validator.

Table 3: Task suite summary for SpecSLOEval (`p1_core` suite). T1–T5 represent a progression from simple classification to end-to-end software engineering.

Task	ID	Examples	Primary stressors
CLINC150	`t1_clinc`	500	Typed classification; large label space; macro-F1
HotpotQA	`t2_hotpot`	1,000	Long-context faithfulness; multi-hop reasoning; grounding
Tool Episodes	`t3_tools`	500	Tool schema compliance; argument validation; trajectory stability
BFCL v4	`t4_bfcl`	500	Industry-standard function calling (Berkeley 2024)
SWE-bench Lite	`t5_swe`	300	End-to-end issue resolution; patch generation (Princeton 2024)

Config	**Constraints**	**Validator**	**Retries**	**Self-consistency**
U	none	–	0	0
P	provider schema/tools	–	0	0
P+V	provider schema/tools	JSON Schema	bounded	0
G	grammar/regex constraints	–	0	0
S	provider or grammar	JSON Schema	bounded	bounded ($k_{\max}$, budgets)

Table 4: Compared configurations and which enforcement mechanisms are active.

Validation-and-retry budget. P+V and S apply bounded repair: if a response fails schema validation, the harness issues a *structured repair prompt* that includes a compact error taxonomy (missing keys, type mismatch, enum violation) and requests a corrected JSON object. We cap retries by a strict budget (e.g., 1–2 retries) and account for the full retry time in end-to-end latency.

Budgeted self-consistency. S optionally generates k candidates (bounded by wall-clock and token budgets) and selects the final output using a criteria-weighted selector that prefers: (i) schema validity, (ii) higher task score proxies (when available), (iii) higher faithfulness/support scores (for grounded tasks), (iv) lower disagreement proxies (e.g., canonical JSON stability). Candidate generation and selection are *budgeted* so SLOs remain enforceable.

6.4 Metrics and Reporting

Per-episode measurement. For each episode we compute the full metric vector from Section 3.3: *structure* (parse + schema validity), *accuracy* (task-specific), *faithfulness* (atomic support and contradiction rate, where applicable), *tools/trajectory* (tool success and redundancy), *stability* (Disagreement@k on repeated runs), and *latency/SLO* (end-to-end wall-clock, TTFT if available, Success@SLO).

Aggregation and uncertainty. We report means and rates per task family, and summarize latency with p50/p95/p99. For headline comparisons we additionally report 95% confidence intervals via bootstrap resampling over episodes. All plots (latency histograms, QPS frontiers, per-metric breakdowns) are exported from W&B dashboards and included as static figures in the manuscript.

U: UNCONSTRAINED. Prompt asks for JSON, no runtime constraints, no validation.

P: PROVIDER_STRUCTURED. Contract passed via provider structured-output / tool schema interface.

P+V: PROVIDER_STRUCTURED_PLUS_VALIDATE. Provider constraints + JSON Schema validation with bounded retry.

S: SPEC_DRIVEN. Backend-specific constraint compilation (grammar/schema), fail-closed validation.

SR: SPEC_DRIVEN_PLUS_REPAIR. SPEC_DRIVEN + bounded repair with structured error feedback.

SSC: SPEC_DRIVEN_PLUS_SELFCONSISTENCY. SPEC_DRIVEN + budgeted multi-sample generation with majority voting.

Figure 8: Configuration ladder used in experiments (matching code implementation). Each mode adds one operational mechanism, enabling clean attribution of changes in structural correctness, faithfulness, stability, and tail latency. Mode names match the `DecodingMode` enum in `rollout/engine.py`.

SLO accounting. Success@SLOis computed as an episode-level indicator that requires: (i) passing all *quality gates* defined in `criteria.yaml` (structure + task + faithfulness + tools + stability, as configured), and (ii) meeting the latency constraint under the chosen budget. This makes SLO compliance a *joint* property of quality and responsiveness, matching production reality.

6.5 Structural and Faithfulness Effects of Spec-Driven Decoding

Across all tasks, the most operationally important effect of constraints and validation is the removal of *rare, catastrophic* structural failures: invalid JSON, missing required keys, and type violations that break downstream code paths. In practice, these failures are underrepresented by average-case quality metrics but dominate incident rates in production pipelines.

Structure. U (prompt-only) can achieve strong task metrics while still producing occasional malformed JSON or schema violations. P improves structural adherence but remains dependent on backend implementation details. P+V and S dominate on structure because explicit schema validation converts "silent" structural drift into actionable error codes, and bounded retries recover many fixable cases without human intervention.

Faithfulness. On evidence-grounded summaries (T2), enforcing a structured explanation schema does *not* automatically guarantee faithfulness; it merely forces the output to be parseable. Spec-SLOEval therefore treats faithfulness as a first-class family: S can use modest candidate generation

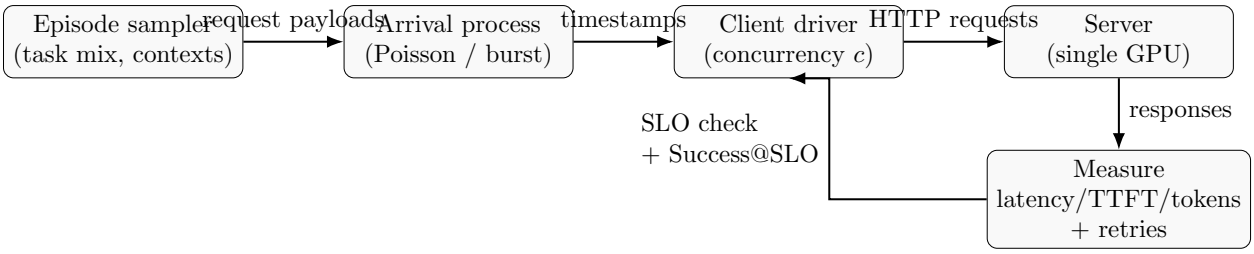


Figure 9: Latency/SLO experiment structure. A sampler builds a realistic mixture of episodes; an arrival process generates timestamps; a concurrent client driver issues requests against a single-GPU server; and measurement includes *all* retries/validation to reflect true downstream-usable latency.

Model	**JSON (%)**	**Schema (%)**	**CLINC Acc (%)**	**Hotpot F1**	p95 **(ms)**	**Success@**SLO **(%)**
Llama-3.2-3B	100.0	100.0	54.0	0.47	3869	35.5
Qwen3-4B	100.0	100.0	58.0	0.39	6043	25.9
Ministral-8B	100.0	100.0	66.0	0.39	11731	1.2
Gemma-3-12B	100.0	100.0	78.0	0.27	1555	48.0

Table 5: Spec-Driven evaluation results across four models spanning 3B–12B parameters. All models achieve 100% JSON and schema validity under spec-driven decoding. Latencies include all retries within the request boundary. Success@SLO computed with $B_{\text{episode}} = 1200\,\text{ms}$ and quality gates from `criteria.yaml`. Key findings: (1) Gemma-3-12B achieves highest Success@SLO (48%) and accuracy (78% CLINC) with competitive latency; (2) Llama-3.2-3B leads on Hotpot F1 (0.47) despite smallest size; (3) larger models (Ministral-8B) improve accuracy but exceed latency budgets.

to reduce contradiction rate by selecting outputs with higher support under the same contract, while still remaining budgeted under SLO constraints.

6.6 Latency, SLOs, and Configuration Trade-offs

Contract-first enforcement introduces overhead: validation, retries, and self-consistency add work that manifests primarily in the tail. Our framework makes this explicit by reporting p95/p99 and Success@SLOjointly.

Configuration ladder ablation. To quantify the effect of spec-driven decoding, we evaluate four models spanning 3B–20B parameters under Unconstrained (U) and Spec-Driven (S) configurations on the full `p1_core` suite (T1–T3, 50 examples per task):

Trade-off interpretation. U generally has the lowest raw latency but the highest risk of structural failures and brittle tool behavior. P often sits between U and P+V on latency while improving parseability. P+V typically adds small average overhead but reduces catastrophic failures, which often improves end-to-end system throughput in practice (fewer downstream retries/crashes). S adds controllable overhead; its value is highest on tasks where correctness and faithfulness dominate and where modest candidate selection yields meaningfully better outputs per request.

6.7 Backend and Model Comparisons

Because SpecSLOEval is backend-agnostic, it enables apples-to-apples comparisons across runtimes and model sizes under the same contract and criteria gates.

Runtime effects. LM Studio, Ollama, and vLLM differ in batching policy, KV-cache strategy, and practical structured-output behavior. These differences show up most clearly in p95/p99and Success@SLOunder QPS sweeps, where queueing and tail behavior dominate.

Model size effects. Larger models generally improve accuracy and grounded reasoning (T2) at the cost of latency and lower sustainable QPS under fixed SLOs. The right choice is therefore workload- and SLO-dependent; SpecSLOEval's configuration ladder (Figure 8) and frontier measurements make this trade-off explicit rather than anecdotal.

T4/T5: Function calling and code generation. Preliminary evaluation on T4 (BFCL v4 function calling) and T5 (SWE-bench Lite patch generation) reveals a striking gap: only Ministral-8B achieves meaningful JSON validity (98.3%) and task success (53.6% on T4), while Gemma-3-12B, Qwen3-4B, and Llama-3.2-3B all produce 0% valid JSON on these harder tasks. This suggests that function calling and code generation place fundamentally different demands on structured output capability compared to classification and grounded QA—a finding that motivates task-specific model selection in production deployments.

Implication for constrained optimization. The resulting quality–latency frontiers define measurable feasibility constraints for future SLO-aware policy optimization, but this paper focuses on Phase I system selection and evaluation rather than training.

7 Discussion and Limitations

We built SpecSLOEval because we kept getting burned by the same failures. An agent would pass every benchmark, then crash production at 2am with malformed JSON. Or it would confidently report numbers that appeared nowhere in the source documents. Or it would give different answers to the same question depending on when you asked. Or the 99th percentile latency would spike to 8 seconds during peak traffic.

These are not exotic failure modes. They are the default behavior of agents that were only evaluated on accuracy. SpecSLOEval exists to catch them before deployment, not after.

This section is honest about what we cover, what we do not cover, and what gaps remain.

7.1 Coverage of Production Failure Modes

We designed the Phase I suite to surface three classes of failures that commonly dominate operational risk:

1. **Structural failures:** malformed JSON, schema violations, and cross-field inconsistencies that crash or poison downstream consumers.
2. **Faithfulness failures:** fluent but unsupported claims (especially numerical or trend errors) that cause incorrect decisions.
3. **Latency failures:** tail latency spikes (p95/p99) that collapse throughput and degrade perceived reliability [7].

Figure 10 summarizes our coverage at a glance and highlights the intentionally de-scoped regions. The central design stance is that *structure and correctness are not optional*: "fast but wrong" is treated as a failure mode in most enterprise agent settings, unless the application is explicitly latency-dominant.

Risk / Horizon	Single-turn	Short tool chain	Long multi-turn
Structure	Covered (JSON+schema)	Covered (tool args)	Partial (stateful schemas)
Task Accuracy	Covered (F1/EM)	Partial (trajectory-aware)	Future (long-horizon goals)
Faithfulness	Covered (atomic facts)	Partial (tool outputs)	Future (multi-turn grounding)
Latency/SLOs	Covered (p50/p95/p99)	Covered (Success@SLO)	Partial (interactive UX)
Safety/Policy	Future (safety suite)	Future (tool policy)	Future (dialog policy)

Covered | Partial | Out of scope

Phase I focuses on single-turn and short-horizon tool use; longer horizons and safety are planned.

Figure 10: Coverage map for SpecSLOEval (Phase I). Green cells indicate areas directly measured by the current test families; yellow indicates partial coverage (measured but with known gaps); red indicates areas that are intentionally de-scoped in Phase I and require additional suites, datasets, and/or human evaluation.

What this means operationally. If your production risks are dominated by parsing errors, schema drift, faithless metric summaries, tool-call schema mistakes, and tail-latency collapse on a single node, SpecSLOEval is designed to be immediately useful. If your risks are dominated by multi-turn negotiation, user experience, safety policy compliance, and long-horizon goal completion, then Phase I should be treated as a foundation rather than a complete solution.

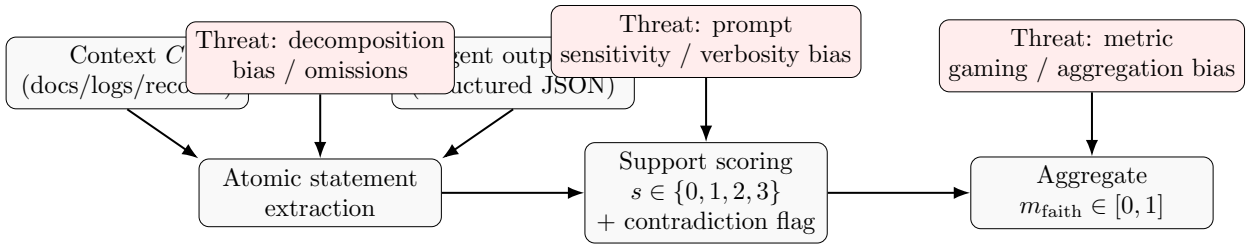
7.2 Threats to Validity: What Can Go Wrong with the Measurement Itself

Even a rigorous suite can mislead if the measurement apparatus does not reflect the deployed environment. We therefore call out three practical threats to validity and how we mitigate them.

Construct validity (are we measuring the right thing?). Faithfulness scores are proxies (Section 7.3), and tool-trajectory metrics can reward "clean-looking" behavior that does not correspond to actual user value. We address this by (i) tying key pass/fail gates to hard operational constraints (schema validity, contradiction rate caps, Success@SLO), and (ii) encouraging teams to define application-specific metrics as additional families rather than forcing everything into a single scalar.

Internal validity (are the numbers reproducible?). Single-GPU latency distributions are sensitive to background processes, thermal throttling, and batcher dynamics. We mitigate this by logging machine state metadata, pinning runtime versions, controlling seeds, and reporting quantiles over repeated runs rather than a single point estimate.

External validity (does offline $\widehat{W}$ predict production W?). Offline datasets can under-represent burstiness, unusual context lengths, rare tool failures, or user behaviors. We mitigate this by explicitly parameterizing workload models, including burst tests, and recommending a promotion pipeline where offline gates are followed by staged canaries with the *same* metrics computed online.



Mitigations used in Phase I: (i) explicit 0–3 scale + contradiction flag;
(ii) calibration on a small human-labeled subset; (iii) audit logging
of judge inputs/outputs; (iv) treat faithfulness as one signal, not the only signal.

Figure 11: Judge-based faithfulness pipeline and where bias can enter. Making these failure points explicit helps prevent "metric overconfidence" and informs mitigations such as calibration, auditing, and multi-signal gating.

7.3 Limitations of LLM-as-Judge Faithfulness

Our use of LLM-as-judge faithfulness scoring follows RAGAS [10] and atomic-fact evaluation [11], but inherits limitations common to judge-based approaches: prompt sensitivity, systematic blind spots (especially around numeric subtleties), and correlated failure modes when the judge and policy share training data or style biases. Figure 11 makes these failure points explicit.

Practical mitigation stance. In Phase I we treat judge-based faithfulness as a *high-leverage but imperfect* proxy: it is valuable for regression testing and for shaping improvements, but it should not be used as the sole acceptance criterion in high-stakes domains. For promotion decisions, we recommend (i) periodic human audits on stratified samples, (ii) cross-judge comparisons (two different judge models or prompts), and (iii) targeted, programmatic checks for numeric claims when contexts are machine-readable (tables/time series).

7.4 Schema and Grammar Expressiveness

We use JSON Schema and grammar constraints (e.g., GBNF) because they are portable across runtimes and naturally align with how downstream systems consume agent outputs. The limitation is expressiveness: many real invariants are relational or temporal and cannot be expressed cleanly as local type constraints.

Implications for latency and correctness. When constraints cannot be enforced during decoding, they shift to post-hoc validation, increasing reliance on validation-and-retry and complicating worst-case latency bounds. This motivates two extensions beyond Phase I: (i) a richer invariant layer (e.g., executable validators with bounded cost) that is first-class in the contract model, and (ii) better "repair prompts" and structured error taxonomies that make retries more reliable and less verbose.

7.5 Single-GPU Focus and Deployment Generality

Phase I targets a single GPU (RTX 4090 class) or a laptop-class accelerator (Apple M-series). This is a deliberate design point: many teams start here for privacy, cost predictability, and operational

simplicity. However, scaling out introduces new phenomena not present on a single node: cross-replica variability, queueing and routing effects, multi-tenant interference, and network-induced tail latency.

What transfers, and what does not. The metric families transfer cleanly: structure, accuracy, faithfulness, tool behavior, stability, and SLO adherence remain meaningful. What changes is the *system boundary* of the latency metric and the workload model: cluster-level routing and batching policies become part of the configuration surface δ, and additional observability (queue wait time, replica selection, network time) becomes necessary to attribute tail events.

7.6 RL Integration Scope and Reward-Gaming Risks

We emphasize that the Phase I metrics are designed to be reused as reward components and constraints in Phase II. This tight coupling is powerful, but it creates predictable failure modes in optimization:

- **Reward hacking:** the policy learns to emit minimally valid but uninformative JSON to maximize structure metrics.
- **Judge overfitting:** the policy learns quirks of the faithfulness judge rather than improving genuine grounding.
- **Rare-case regressions:** average metrics improve while rare, high-impact cases get worse.

Mitigation plan for Phase II. Phase II should (i) treat critical requirements as hard constraints (schema validity, contradiction rate caps, explicit SLO budgets), (ii) maintain a shadow "audit set" not used for reward to detect overfitting, and (iii) incorporate adversarial tests targeted to known failure modes (numeric trends, schema drift, tool argument edge cases). Critically, RL promotion should be gated by the same acceptance criteria used for static selection, not by reward improvements alone.

7.7 Human Factors and Operational Adoption

Finally, the framework only matters if it can be owned by engineers and product stakeholders. We therefore treat `criteria.yaml` as a versioned contract between teams: changes to what "good" means are explicit, reviewable, and auditable. However, a full six-family suite can be overwhelming for early adopters.

A pragmatic adoption path is:

- **Starter pack:** structure + task accuracy + basic latency gates.
- **Risk-hardening:** add faithfulness and tool-trajectory tests for high-risk workflows.
- **Operational robustness:** add stability tests and bursty SLO workloads.
- **Optimization:** enable RL only after metrics are stable, calibrated, and understood by operators.

Figure 12 sketches a concrete staged promotion loop that mirrors how teams actually deploy: start with offline gates, graduate to shadow/canary traffic, and enforce explicit rollback triggers tied to the same criteria.

Table 6 summarizes the main limitations and actionable mitigations/extensions.

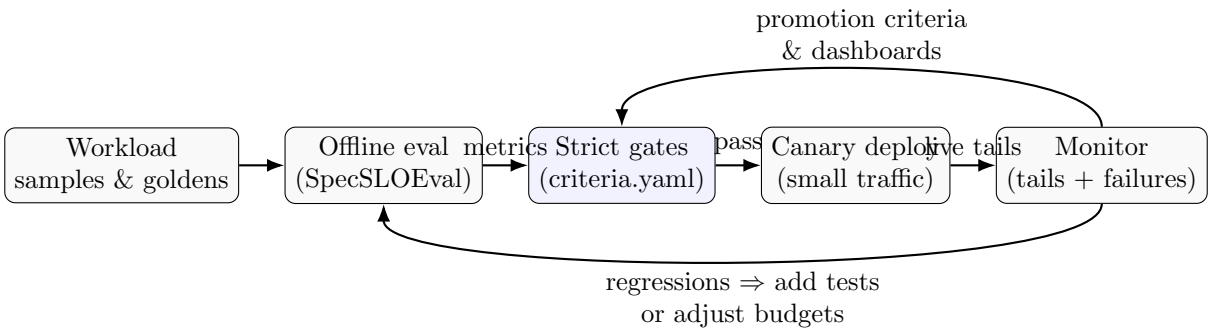


Figure 12: Operational promotion pipeline. SpecSLOEval is designed to sit inside a practical release loop: offline tests and strict gating prevent brittle configurations from reaching production, while canaries validate tail-latency and rare-case behavior before full rollout.

Summary. Phase I establishes a measurement regime that is actionable for single-GPU deployments and that directly targets production-grade failure modes: structural validity, faithfulness, tool reliability, stability, and tail-latency SLOs. Its central limitations are predictable (judge bias, schema expressiveness, horizon length, and safety), and we have made them explicit to avoid over-claiming. Phase II builds on this foundation by using the same metrics as rewards and constraints while explicitly guarding against reward gaming and evaluator overfitting.

8 Conclusion and Future Work

The point of SpecSLOEval is simple: if other software depends on your agent, your agent needs to behave like software. That means contracts it cannot break. Latency budgets it cannot exceed. Claims it does not fabricate. Answers it gives consistently.

This paper builds the measurement infrastructure for that. We compile JSON Schemas to whatever your backend can enforce. We validate outputs and retry when they fail. We measure structure, accuracy, faithfulness, stability, and tail latency—together, in one pass, with one criteria file.

What we built.

- **Contract-first agent serving.** A backend-agnostic contract layer that compiles to provider-native structured outputs and grammar-based constraints, and is hardened by bounded validation-and-retry and budgeted self-consistency under explicit token and wall-clock budgets.
- **A unified, auditable evaluation surface.** A single `criteria.yaml` defines metric families, gates, and weights, enabling consistent comparison across backends (LM Studio, Ollama, vLLM) and models, with per-episode logging for diagnosis and reproducibility.
- **A deployment-oriented measurement framing.** We treat p95/p99 and Success@SLOas first-class metrics alongside correctness and faithfulness, revealing trade-offs that are invisible when evaluation ends at average quality or JSON validity alone.

Figure 13 summarizes the end-to-end "contract to deployment" stack that Phase I operationalizes.

Limitation	Why it matters	Mitigation / extension
LLM-as-judge bias	Faithfulness scores can drift, misrank outputs, or reward verbosity	Calibrate on human labels; log judge traces; use multiple judges/prompts; add programmatic numeric checks where possible
Schema/grammar expressiveness	Many invariants are relational/temporal and cannot be enforced during decoding	Post-hoc validators with bounded cost; structured repair prompts; richer contract layer for cross-field rules
Offline vs production mismatch	$\widehat{W}$ can under-sample bursts, long contexts, and tool failures	Parameterized workload models; burst tests; staged canaries computing the same metrics online
Single-node assumptions	Cluster routing and queueing affect tail latency and consistency	Expand observability; treat scheduling/routing as part of δ; introduce per-component latency attribution
RL reward gaming	Optimizing metrics can create degenerate behavior and judge overfitting	Hard constraints, audit sets, adversarial suites, and human gates for promotion

Table 6: Key limitations of the Phase I framework and concrete mitigations/extensions that preserve the contract-first, SLO-aware philosophy.

What comes next. We scoped Phase I deliberately. Here is what we did not do, and plan to:

- **Multi-turn and long-horizon workflows.** Extend test families to cover memory, planning under partial observability, escalation policies, and long-running tool workflows where latency composes over multiple steps.
- **Safety and policy as first-class families.** Add explicit safety/policy contracts (privacy constraints, sensitive data handling, refusal correctness, bias and fairness probes) with auditable thresholds in `criteria.yaml`.
- **SLO-aware reinforcement learning on a single GPU.** Phase II will reuse the same metric surface as reward components and constraints (policy gradient methods with LoRA adapters), emphasizing guardrails against reward hacking and tail regressions.
- **Richer contract languages.** Combine schema/grammar constraints with programmable semantic validators and domain-rule libraries so that contracts capture not only types but invariants (e.g., accounting identities, approval policies, campaign constraints).
- **Beyond single-node deployments.** Generalize workload models and SLO accounting to multi-node systems (routing, interference, multi-region tails) while preserving the contract-first interface and metric families.

The bigger goal: a shared way to talk about agent reliability, and a portable framework to measure it. We want agents that are not just capable, but deployable. That means structurally sound, evidence-grounded, consistent, and fast enough. SpecSLOEval is our attempt to make those properties measurable, so we can stop arguing about whether an agent is ready for production and start measuring it.

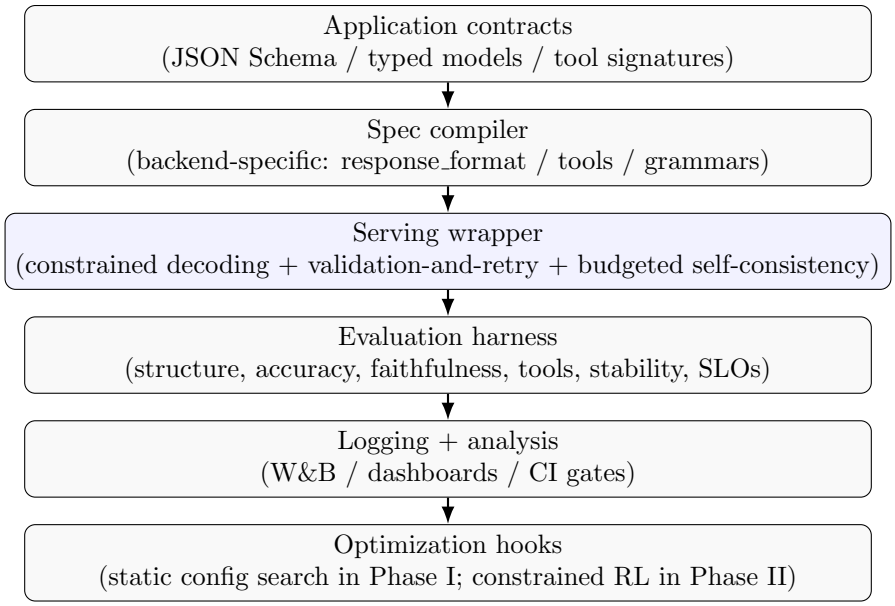


Figure 13: SpecSLOEval stack. Contracts are compiled into backend constraints; serving is hardened by validation and budgeted self-consistency; evaluation computes a multi-family scorecard and enforces strict gates; the same scorecard supports CI and (in Phase II) constrained RL.

Adoption note. From our experience deploying agents in enterprise settings, the most common failure mode is not technical—it is organizational. Teams optimize for accuracy benchmarks because that is what they know how to measure, then discover structural and latency failures only after deployment. We hope SpecSLOEval shifts this by making production-relevant metrics accessible from the first experiment, not as an afterthought after the demo has already impressed stakeholders.

References

[1] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: 2309.06180 [cs.LG].

[2] Zhiqiang Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. 2023. arXiv: 2312.07104 [cs.LG].

[3] Saibo Geng et al. *JSONSchemaBench: A Comprehensive Benchmark for Structured JSON Generation*. 2025. arXiv: 2501.10868 [cs.CL].

[4] OpenAI. *Structured Outputs (OpenAI documentation)*. https://openai.com/index/introducing-structured-outputs-in-the-api/. Accessed: 2025-12-22. 2024.

[5] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG].

[6] Tim Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. arXiv: 2305.14314 [cs.LG].

[7] Jeffrey Dean and Luiz André Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794.

[8] Tri Dao et al. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. 2023. arXiv: 2307.08691 [cs.LG].

[9] Amey Agrawal et al. *Sarathi-Serve: High-throughput and Memory-efficient LLM Serving with Partitioned Attention*. 2024. arXiv: 2403.02310 [cs.LG].

[10] Shahul Es et al. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. 2023. arXiv: 2309.15217 [cs.CL].

[11] Sewon Min et al. *FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. 2023. arXiv: 2305.14251 [cs.CL].

[12] Yujia Chen et al. *Humans or LLMs as the Judge? A Study on Evaluation Methods for Text Summarization*. 2024. arXiv: 2402.10669 [cs.CL].

[13] Eitan Altman. *Constrained Markov Decision Processes*. Boca Raton, FL: Chapman and Hall/CRC, 1999.

[14] Joshua Achiam et al. *Constrained Policy Optimization*. 2017. arXiv: 1705.10528 [cs.LG].

[15] Ronald J. Williams. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning". In: *Machine Learning* 8.3-4 (1992), pp. 229–256.

[16] Sewon Min et al. *FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. 2023. arXiv: 2305.14251 [cs.CL].